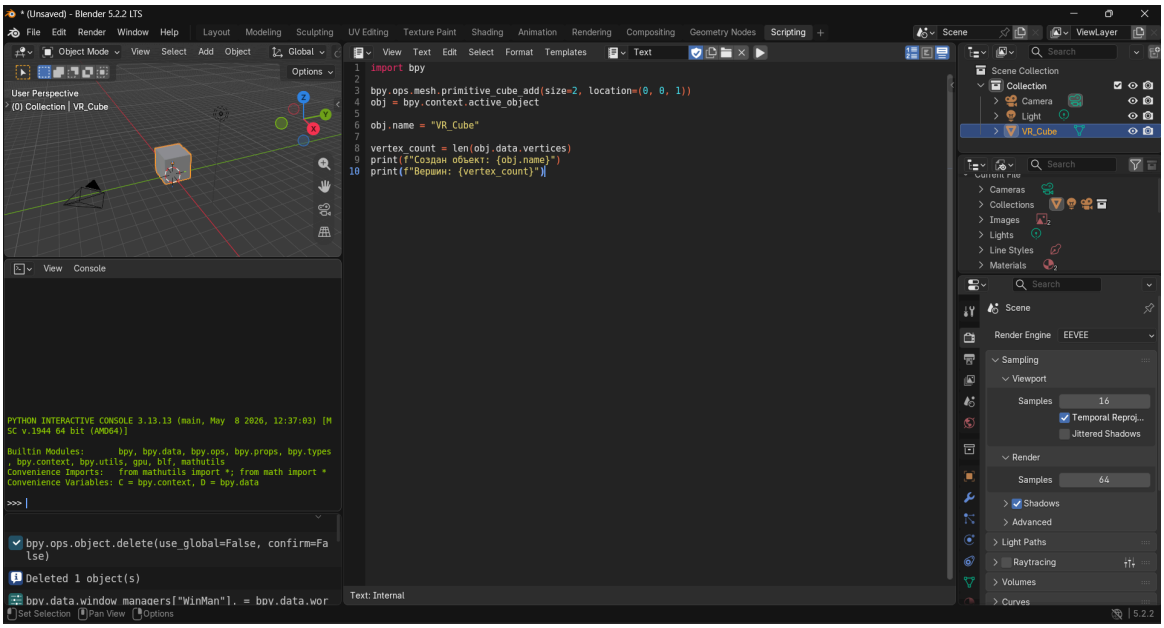
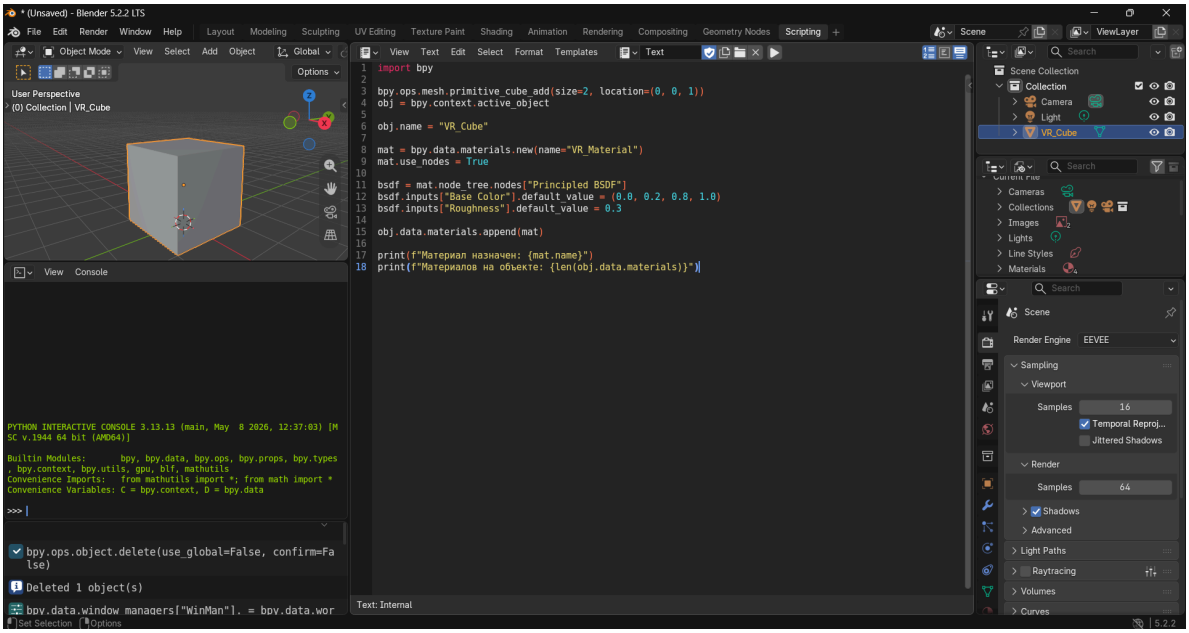
VR Laba3

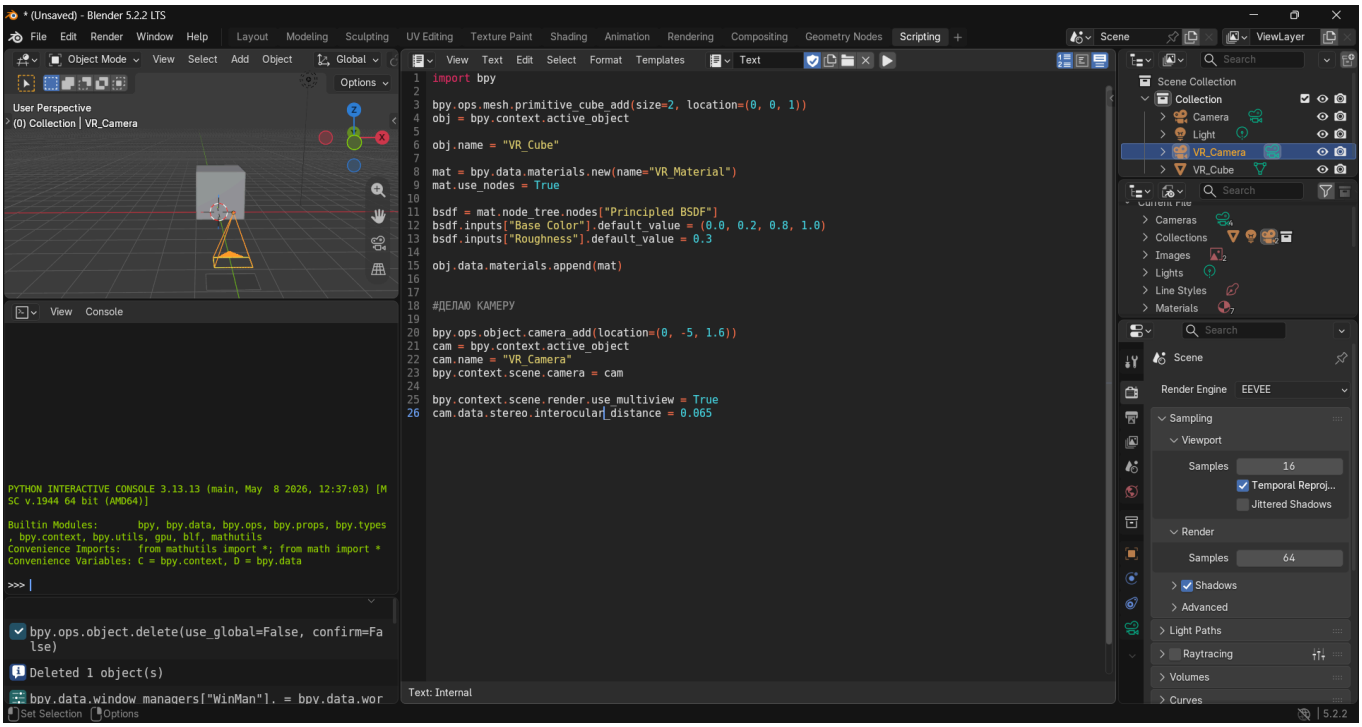
Task 1.



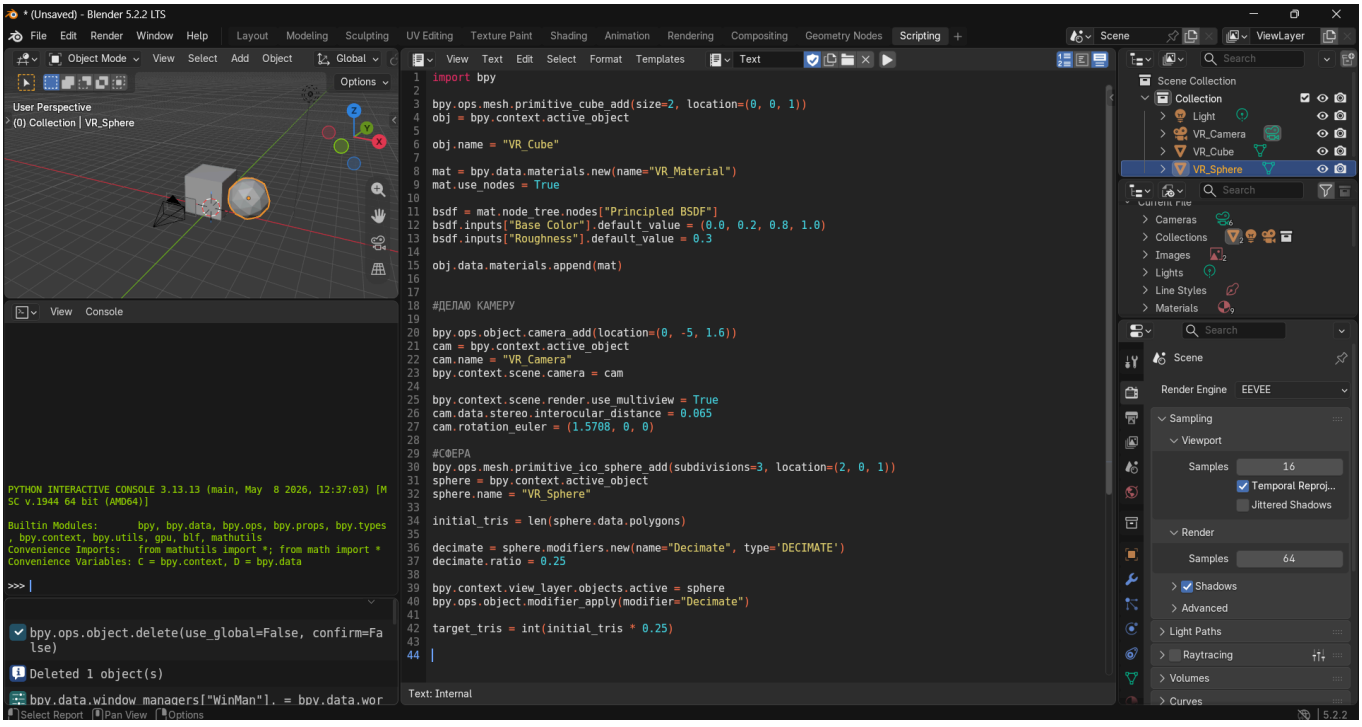
Task 2



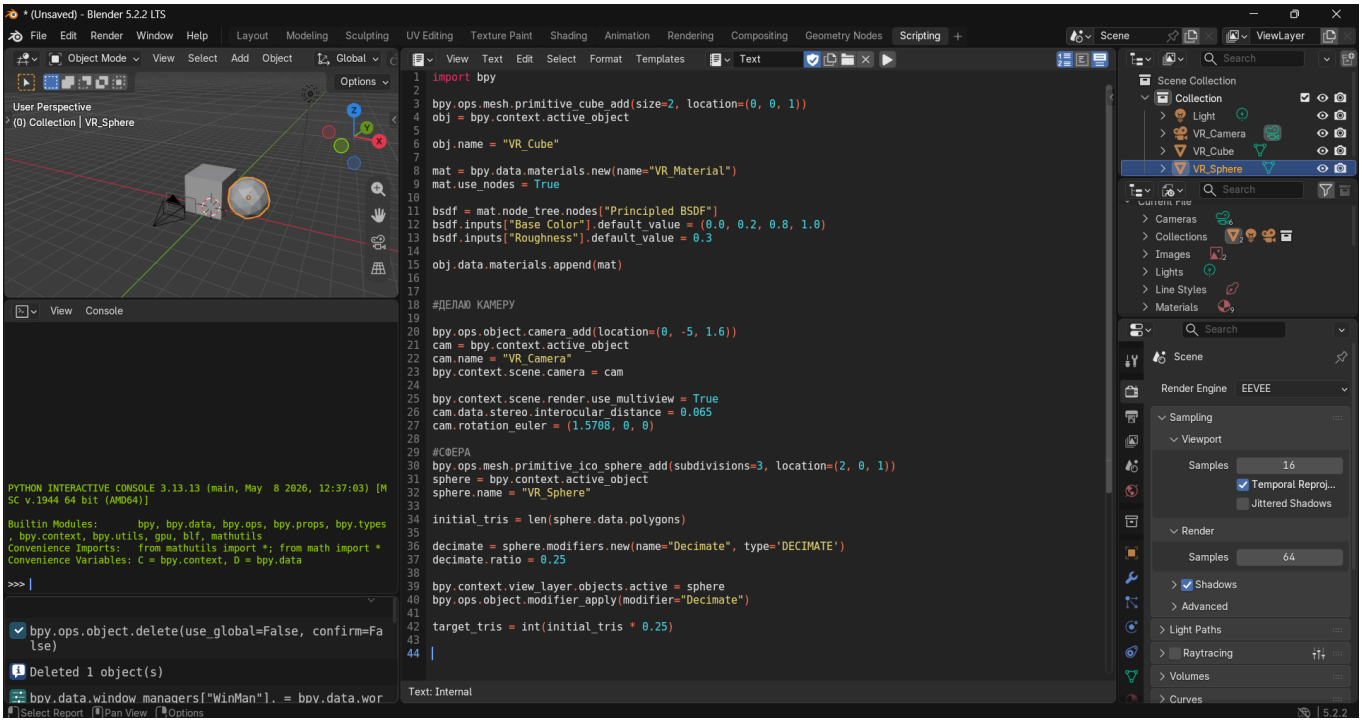
Task3


* (Unsaved) - Blender 5.2.2 LTS
File Edit Render Window Help
Layout Modeling Sculpting UV Editing Texture Paint Shading Animation Rendering Compositing Geometry Nodes Scripting
Scene
ViewLayer
Object Mode View Select Add Object
Global
Options
User Perspective
(0) Collection | VR_Camera
View Console
PYTHON INTERACTIVE CONSOLE 3.13.13 (main, May 8 2026, 12:37:03) [MSC v.1944 64 bit (AMD64)]
Builtin Modules: bpy, bpy.data, bpy.ops, bpy.props, bpy.types, bpy.context, bpy.utils, gpu, blf, mathutils
Convenience Imports: from mathutils import *; from math import *
Convenience Variables: C = bpy.context, D = bpy.data
>>>
bpy.ops.object.delete(use_global=False, confirm=False)
Deleted 1 object(s)
bpy.data.window_managers["WinMan"]. = bpy.data.wor
Set Selection Options
View Text Edit Select Format Templates
Text
import bpy
bpy.ops.mesh.primitive_cube_add(size=2, location=(0, 0, 1))
obj = bpy.context.active_object
obj.name = "VR_Cube"
mat = bpy.data.materials.new(name="VR_Material")
mat.use_nodes = True
bsdf = mat.node_tree.nodes["Principled BSDF"]
bsdf.inputs["Base Color"].default_value = (0.0, 0.2, 0.8, 1.0)
bsdf.inputs["Roughness"].default_value = 0.3
obj.data.materials.append(mat)
#ДЕЛАЮ КАМЕРУ
bpy.ops.object.camera_add(location=(0, -5, 1.6))
cam = bpy.context.active_object
cam.name = "VR_Camera"
bpy.context.scene.camera = cam
bpy.context.scene.render.use_multiview = True
cam.data.stereo.interocular_distance = 0.065
Text: Internal
Scene Collection
Collection
Camera
Light
VR_Camera
VR_Cube
Cameras
Collections
Images
Lights
Line Styles
Materials
Scene
Render Engine EEVEE
Sampling
Viewport
Samples 16
Temporal Reproj...
Jittered Shadows
Render
Samples 64
Shadows
Advanced
Light Paths
Raytracing
Volumes
Curves
5.2.2

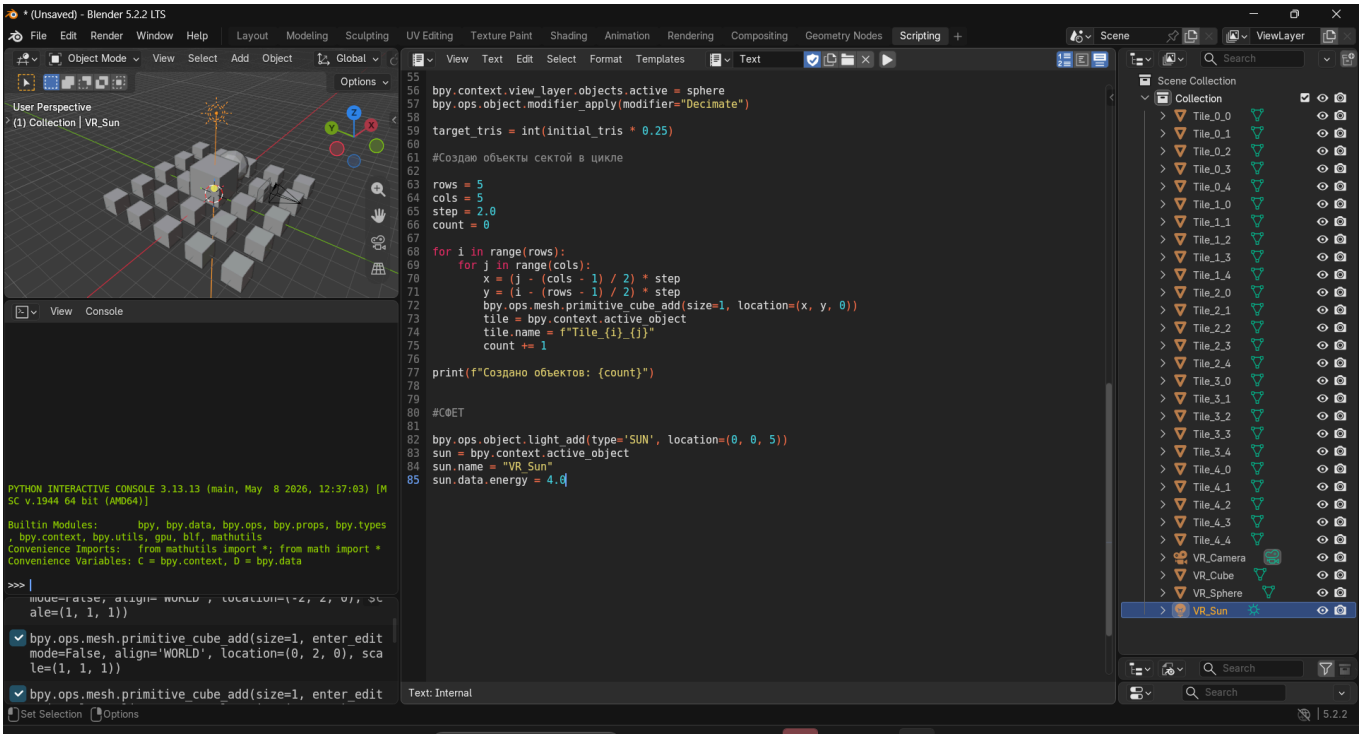
Task4


* (Unsaved) - Blender 5.2.2 LTS
File Edit Render Window Help
Layout Modeling Sculpting UV Editing Texture Paint Shading Animation Rendering Compositing Geometry Nodes Scripting
Scene
ViewLayer
Object Mode View Select Add Object
Global
Options
User Perspective
(0) Collection | VR_Sphere
View Console
PYTHON INTERACTIVE CONSOLE 3.13.13 (main, May 8 2026, 12:37:03) [MSC v.1944 64 bit (AMD64)]
Builtin Modules: bpy, bpy.data, bpy.ops, bpy.props, bpy.types, bpy.context, bpy.utils, gpu, blf, mathutils
Convenience Imports: from mathutils import *; from math import *
Convenience Variables: C = bpy.context, D = bpy.data
>>>
bpy.ops.object.delete(use_global=False, confirm=False)
Deleted 1 object(s)
bpy.data.window_managers["WinMan"]. = bpy.data.wor
Select Report Pan View Options
View Text Edit Select Format Templates
Text
import bpy
bpy.ops.mesh.primitive_cube_add(size=2, location=(0, 0, 1))
obj = bpy.context.active_object
obj.name = "VR_Cube"
mat = bpy.data.materials.new(name="VR_Material")
mat.use_nodes = True
bsdf = mat.node_tree.nodes["Principled BSDF"]
bsdf.inputs["Base Color"].default_value = (0.0, 0.2, 0.8, 1.0)
bsdf.inputs["Roughness"].default_value = 0.3
obj.data.materials.append(mat)
#ДЕЛАЮ КАМЕРУ
bpy.ops.object.camera_add(location=(0, -5, 1.6))
cam = bpy.context.active_object
cam.name = "VR_Camera"
bpy.context.scene.camera = cam
bpy.context.scene.render.use_multiview = True
cam.data.stereo.interocular_distance = 0.065
cam.rotation_euler = (1.5708, 0, 0)
#СФЕРА
bpy.ops.mesh.primitive_ico_sphere_add(subdivisions=3, location=(2, 0, 1))
sphere = bpy.context.active_object
sphere.name = "VR_Sphere"
initial_tris = len(sphere.data.polygons)
decimate = sphere.modifiers.new(name="Decimate", type='DECIMATE')
decimate.ratio = 0.25
bpy.context.view_layer.objects.active = sphere
bpy.ops.object.modifier_apply(modifier="Decimate")
target_tris = int(initial_tris * 0.25)
Text: Internal
Scene Collection
Collection
Light
VR_Camera
VR_Cube
VR_Sphere
Cameras
Collections
Images
Lights
Line Styles
Materials
Scene
Render Engine EEVEE
Sampling
Viewport
Samples 16
Temporal Reproj...
Jittered Shadows
Render
Samples 64
Shadows
Advanced
Light Paths
Raytracing
Volumes
Curves
5.2.2

Task5

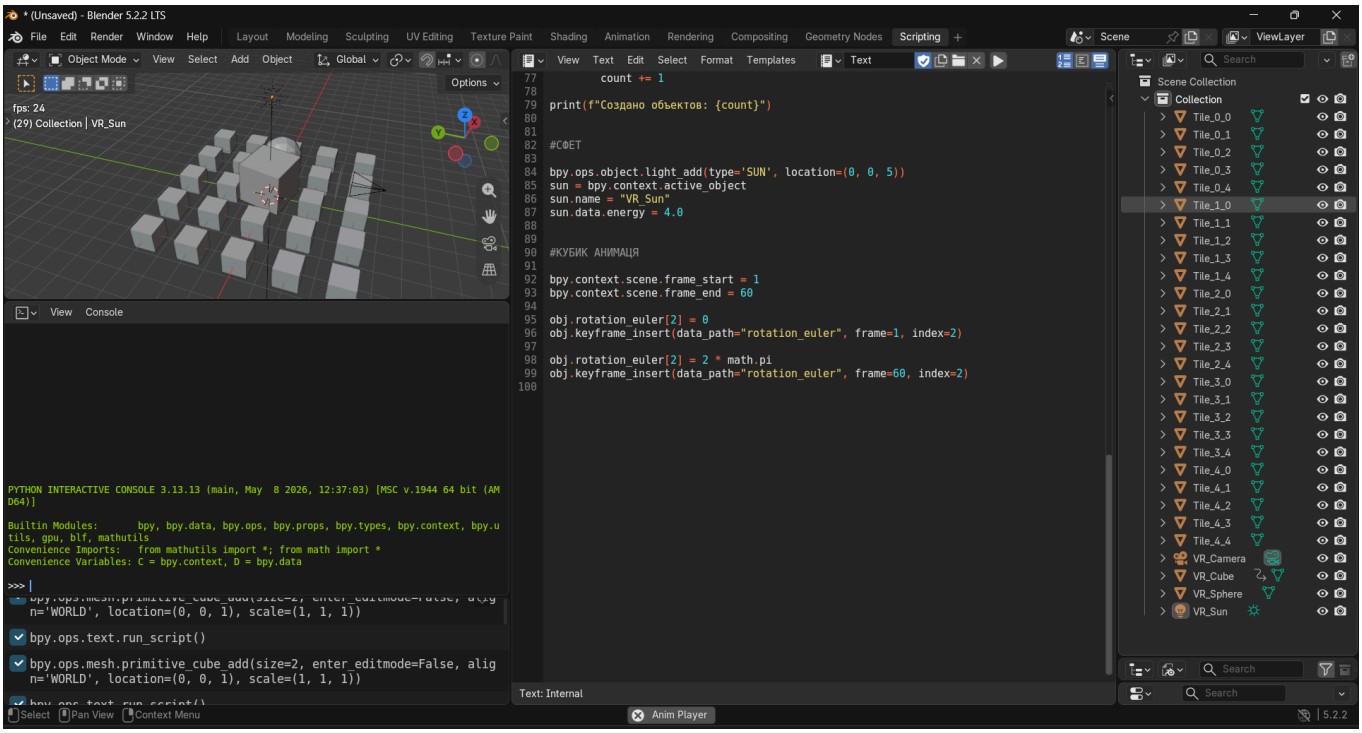


Task6

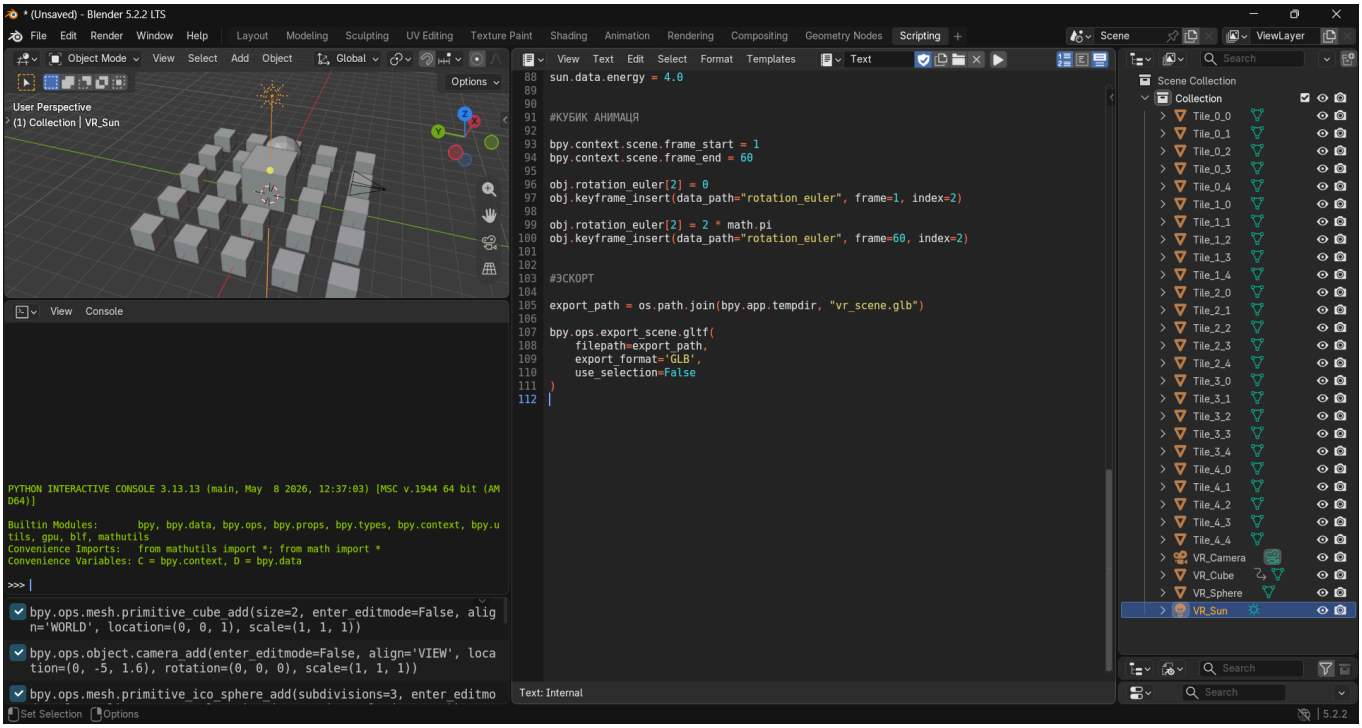


task7

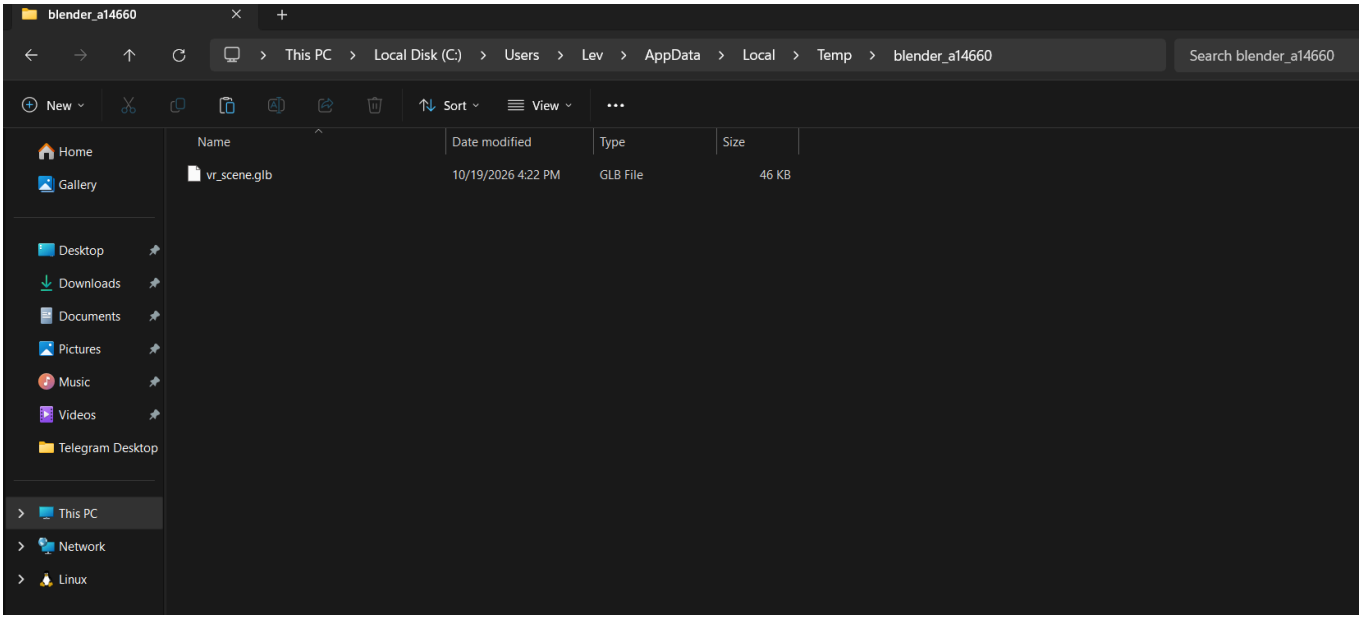



* (Unsaved) - Blender 5.2.2 LTS
File Edit Render Window Help Layout Modeling Sculpting UV Editing Texture Paint Shading Animation Rendering Compositing Geometry Nodes Scripting
Object Mode View Select Add Object Global Options
fps: 24
(29) Collection | VR_Sun
View Text Edit Select Format Templates Text
count += 1
print(f"Создано объектов: {count}")
#СФЕТ
bpy.ops.object.light_add(type='SUN', location=(0, 0, 5))
sun = bpy.context.active_object
sun.name = "VR_Sun"
sun.data.energy = 4.0
#КУБИК АНИМАЦЯ
bpy.context.scene.frame_start = 1
bpy.context.scene.frame_end = 60
obj.rotation_euler[2] = 0
obj.keyframe_insert(data_path="rotation_euler", frame=1, index=2)
obj.rotation_euler[2] = 2 * math.pi
obj.keyframe_insert(data_path="rotation_euler", frame=60, index=2)
Scene Collection
Collection
Tile_0_0
Tile_0_1
Tile_0_2
Tile_0_3
Tile_0_4
Tile_1_0
Tile_1_1
Tile_1_2
Tile_1_3
Tile_1_4
Tile_2_0
Tile_2_1
Tile_2_2
Tile_2_3
Tile_2_4
Tile_3_0
Tile_3_1
Tile_3_2
Tile_3_3
Tile_3_4
Tile_4_0
Tile_4_1
Tile_4_2
Tile_4_3
Tile_4_4
VR_Camera
VR_Cube
VR_Sphere
VR_Sun
View Console
PYTHON INTERACTIVE CONSOLE 3.13.13 (main, May 8 2026, 12:37:03) [MSC v.1944 64 bit (AMD64)]
Builtin Modules: bpy, bpy.data, bpy.ops, bpy.props, bpy.types, bpy.context, bpy.utils, gpu, blf, mathutils
Convenience Imports: from mathutils import *; from math import *
Convenience Variables: C = bpy.context, D = bpy.data
>>>
n='WORLD', location=(0, 0, 1), scale=(1, 1, 1))
bpy.ops.text.run_script()
bpy.ops.mesh.primitive_cube_add(size=2, enter_editmode=False, align='WORLD', location=(0, 0, 1), scale=(1, 1, 1))
Text: Internal
Select Pan View Context Menu
Anim Player
5.2.2

task8


* (Unsaved) - Blender 5.2.2 LTS
File Edit Render Window Help Layout Modeling Sculpting UV Editing Texture Paint Shading Animation Rendering Compositing Geometry Nodes Scripting
Object Mode View Select Add Object Global Options
User Perspective
(1) Collection | VR_Sun
View Text Edit Select Format Templates Text
sun.data.energy = 4.0
#КУБИК АНИМАЦЯ
bpy.context.scene.frame_start = 1
bpy.context.scene.frame_end = 60
obj.rotation_euler[2] = 0
obj.keyframe_insert(data_path="rotation_euler", frame=1, index=2)
obj.rotation_euler[2] = 2 * math.pi
obj.keyframe_insert(data_path="rotation_euler", frame=60, index=2)
#ЭСКОРТ
export_path = os.path.join(bpy.app.tempdir, "vr_scene.glb")
bpy.ops.export_scene.gltf(
filepath=export_path,
export_format='GLB',
use_selection=False
)
Scene Collection
Collection
Tile_0_0
Tile_0_1
Tile_0_2
Tile_0_3
Tile_0_4
Tile_1_0
Tile_1_1
Tile_1_2
Tile_1_3
Tile_1_4
Tile_2_0
Tile_2_1
Tile_2_2
Tile_2_3
Tile_2_4
Tile_3_0
Tile_3_1
Tile_3_2
Tile_3_3
Tile_3_4
Tile_4_0
Tile_4_1
Tile_4_2
Tile_4_3
Tile_4_4
VR_Camera
VR_Cube
VR_Sphere
VR_Sun
View Console
PYTHON INTERACTIVE CONSOLE 3.13.13 (main, May 8 2026, 12:37:03) [MSC v.1944 64 bit (AMD64)]
Builtin Modules: bpy, bpy.data, bpy.ops, bpy.props, bpy.types, bpy.context, bpy.utils, gpu, blf, mathutils
Convenience Imports: from mathutils import *; from math import *
Convenience Variables: C = bpy.context, D = bpy.data
>>>
bpy.ops.mesh.primitive_cube_add(size=2, enter_editmode=False, align='WORLD', location=(0, 0, 1), scale=(1, 1, 1))
bpy.ops.object.camera_add(enter_editmode=False, align='VIEW', location=(0, -5, 1.6), rotation=(0, 0, 0), scale=(1, 1, 1))
bpy.ops.mesh.primitive_ico_sphere_add(subdivisions=3, enter_editmo
Text: Internal
Set Selection Options
5.2.2

**



**

Code:

```
import bpy
import math
import os

# УДАЛЯЮ ВСЕ, ЧТОБ ОН МНЕ ПРИ КАЖДОМ ПЕРЕЗАПУСКЕ НЕ СОЗДАВАЛ 2МЛН НОВЫХ ОБЪЕКТОВ
bpy.ops.object.select_all(action='SELECT')
bpy.ops.object.delete(use_global=True)

for mat in bpy.data.materials:
    bpy.data.materials.remove(mat)
for mesh in bpy.data.meshes:
    bpy.data.meshes.remove(mesh)
for cam in bpy.data.cameras:
    bpy.data.cameras.remove(cam)
for light in bpy.data.lights:
    bpy.data.lights.remove(light)
for action in bpy.data.actions:
    bpy.data.actions.remove(action)

bpy.context.scene.frame_set(1)

#КИБУК
bpy.ops.mesh.primitive_cube_add(size=2, location=(0, 0, 1))
obj = bpy.context.active_object

obj.name = "VR_Cube"
```

```
mat = bpy.data.materials.new(name="VR_Material")
mat.use_nodes = True

bsdf = mat.node_tree.nodes["Principled BSDF"]
bsdf.inputs["Base Color"].default_value = (0.0, 0.2, 0.8, 1.0)
bsdf.inputs["Roughness"].default_value = 0.3

obj.data.materials.append(mat)


#ДЕЛАЮ КАМЕРУ

bpy.ops.object.camera_add(location=(0, -5, 1.6))
cam = bpy.context.active_object
cam.name = "VR_Camera"
bpy.context.scene.camera = cam

bpy.context.scene.render.use_multiview = True
cam.data.stereo.interocular_distance = 0.065
cam.rotation_euler = (1.5708, 0, 0)

#СФЕРА
bpy.ops.mesh.primitive_ico_sphere_add(subdivisions=3, location=(2, 0, 1))
sphere = bpy.context.active_object
sphere.name = "VR_Sphere"

initial_tris = len(sphere.data.polygons)

decimate = sphere.modifiers.new(name="Decimate", type='DECIMATE')
decimate.ratio = 0.25

bpy.context.view_layer.objects.active = sphere
bpy.ops.object.modifier_apply(modifier="Decimate")

target_tris = int(initial_tris * 0.25)

#Создаю объекты сектой в цикле

rows = 5
cols = 5
step = 2.0
count = 0

for i in range(rows):
    for j in range(cols):
```

```
        x = (j - (cols - 1) / 2) * step
        y = (i - (rows - 1) / 2) * step
        bpy.ops.mesh.primitive_cube_add(size=1, location=(x, y, 0))
        tile = bpy.context.active_object
        tile.name = f"Tile_{i}_{j}"
        count += 1

print(f"Создано объектов: {count}")


#СФЕТ

bpy.ops.object.light_add(type='SUN', location=(0, 0, 5))
sun = bpy.context.active_object
sun.name = "VR_Sun"
sun.data.energy = 4.0


#КУБИК АНИМАЦЯ

bpy.context.scene.frame_start = 1
bpy.context.scene.frame_end = 60

obj.rotation_euler[2] = 0
obj.keyframe_insert(data_path="rotation_euler", frame=1, index=2)

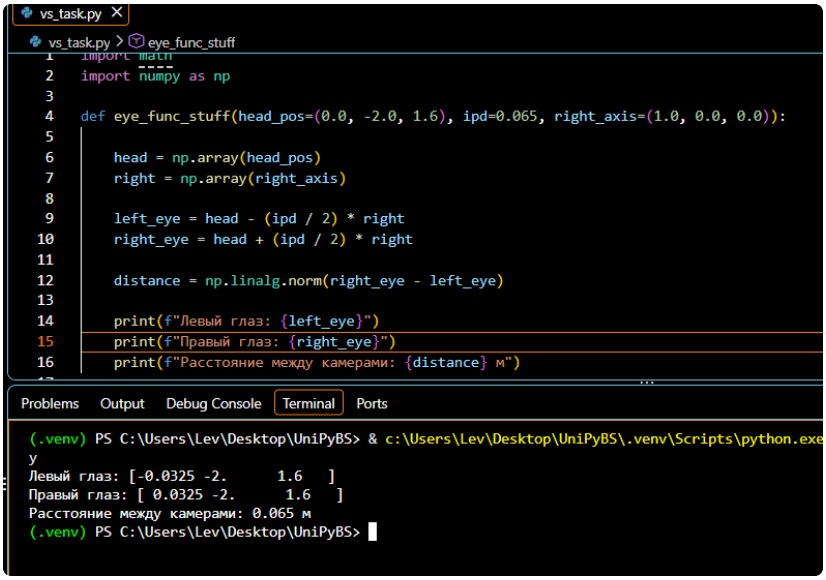
obj.rotation_euler[2] = 2 * math.pi
obj.keyframe_insert(data_path="rotation_euler", frame=60, index=2)


#ЭСКОРТ

export_path = os.path.join(bpy.app.tempdir, "vr_scene.glb")

bpy.ops.export_scene.gltf(
    filepath=export_path,
    export_format='GLB',
    use_selection=False
)
```

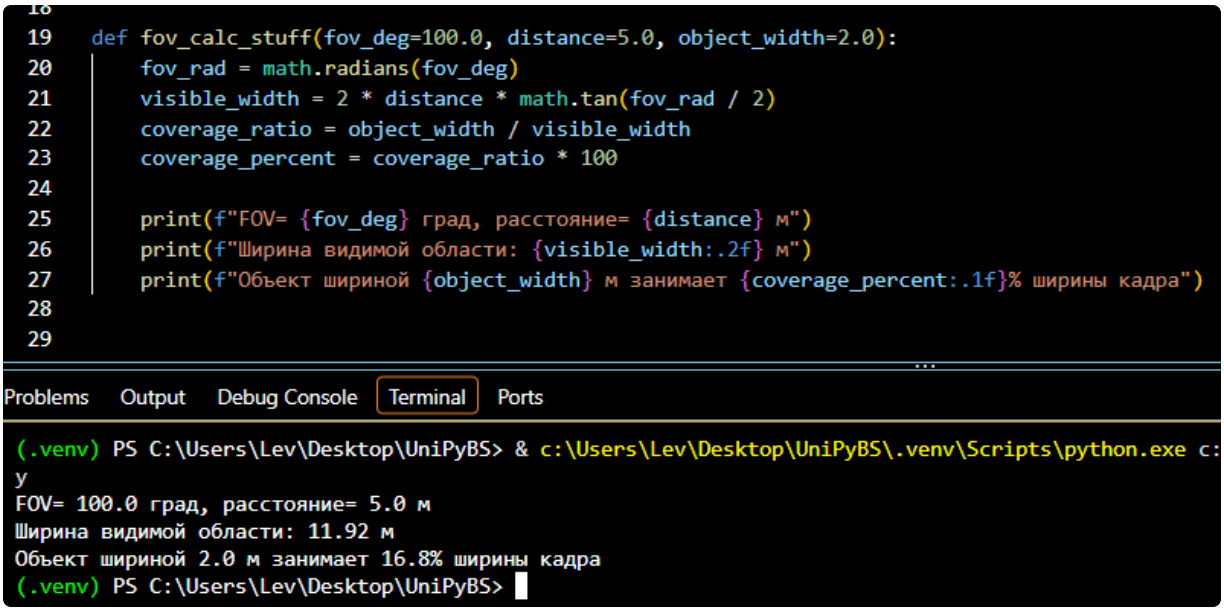
Task 9.



Code:

```
def eye_func_stuff(head_pos=(0.0, -2.0, 1.6), ipd=0.065, right_axis=(1.0, 0.0,
0.0)):
    head = np.array(head_pos)
    right = np.array(right_axis)
    left_eye = head - (ipd / 2) * right
    right_eye = head + (ipd / 2) * right
    distance = np.linalg.norm(right_eye - left_eye)
    print(f"Левый глаз: {left_eye}")
    print(f"Правый глаз: {right_eye}")
    print(f"Расстояние между камерами: {distance} м")
```

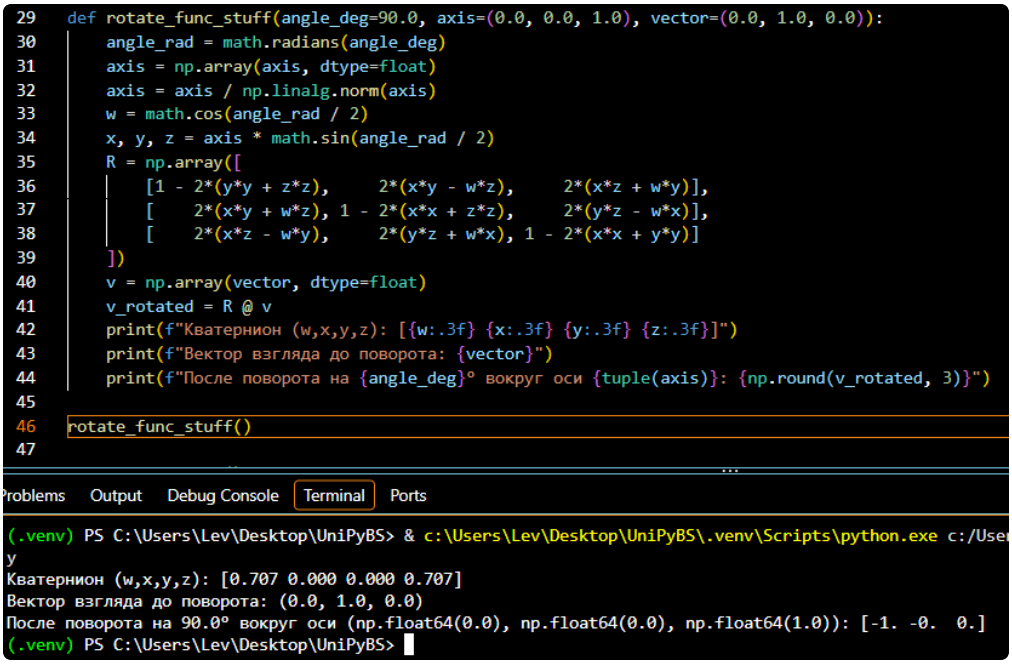
Task 10.



Code:

```python
def fov_calc_stuff(fov_deg=100.0, distance=5.0, object_width=2.0):
    fov_rad = math.radians(fov_deg)
    visible_width = 2 * distance * math.tan(fov_rad / 2)
    coverage_ratio = object_width / visible_width
    coverage_percent = coverage_ratio * 100
    print(f"FOV= {fov_deg} град, расстояние= {distance} м")
    print(f"Ширина видимой области: {visible_width:.2f} м")
    print(f"Объект шириной {object_width} м занимает {coverage_percent:.1f}%
ширины кадра")
```

Task11.

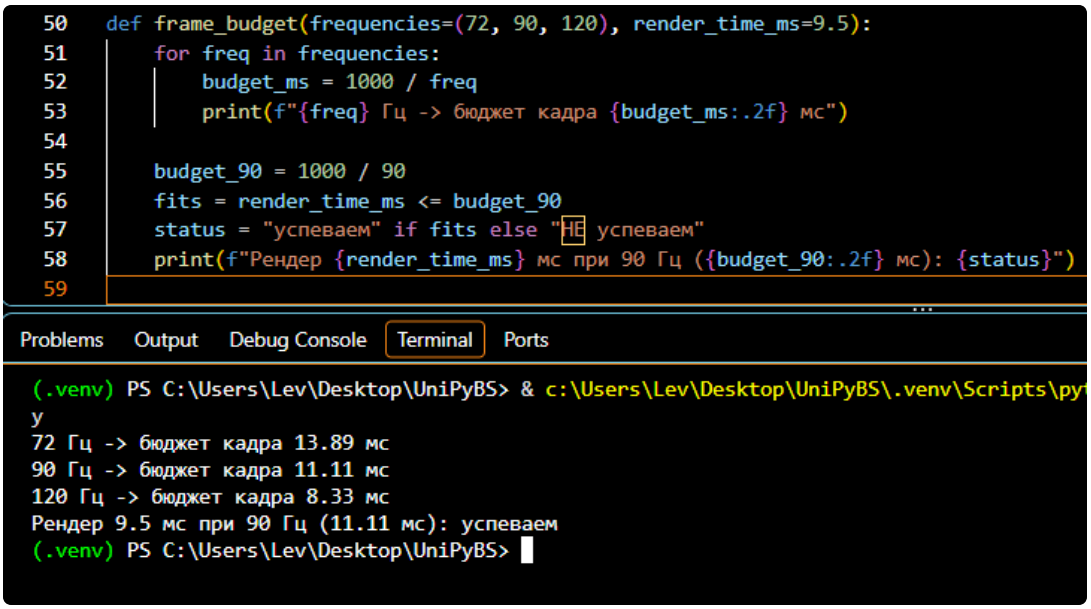


Code:

```python
def rotate_func_stuff(angle_deg=90.0, axis=(0.0, 0.0, 1.0), vector=(0.0, 1.0,
0.0)):
    angle_rad = math.radians(angle_deg)
    axis = np.array(axis, dtype=float)
    axis = axis / np.linalg.norm(axis)
    w = math.cos(angle_rad / 2)
    x, y, z = axis * math.sin(angle_rad / 2)
    R = np.array([
        [1 - 2*(y*y + z*z),     2*(x*y - w*z),     2*(x*z + w*y)],
        [    2*(x*y + w*z), 1 - 2*(x*x + z*z),     2*(y*z - w*x)],
        [    2*(x*z - w*y),     2*(y*z + w*x), 1 - 2*(x*x + y*y)]
    ])
    v = np.array(vector, dtype=float)
    v_rotated = R @ v
    print(f"Кватернион (w,x,y,z): [{w:.3f} {x:.3f} {y:.3f} {z:.3f}]")
    print(f"Вектор взгляда до поворота: {vector}")
```

```
    print(f"После поворота на {angle_deg}° вокруг оси {tuple(axis)}:
{np.round(v_rotated, 3)}")
```

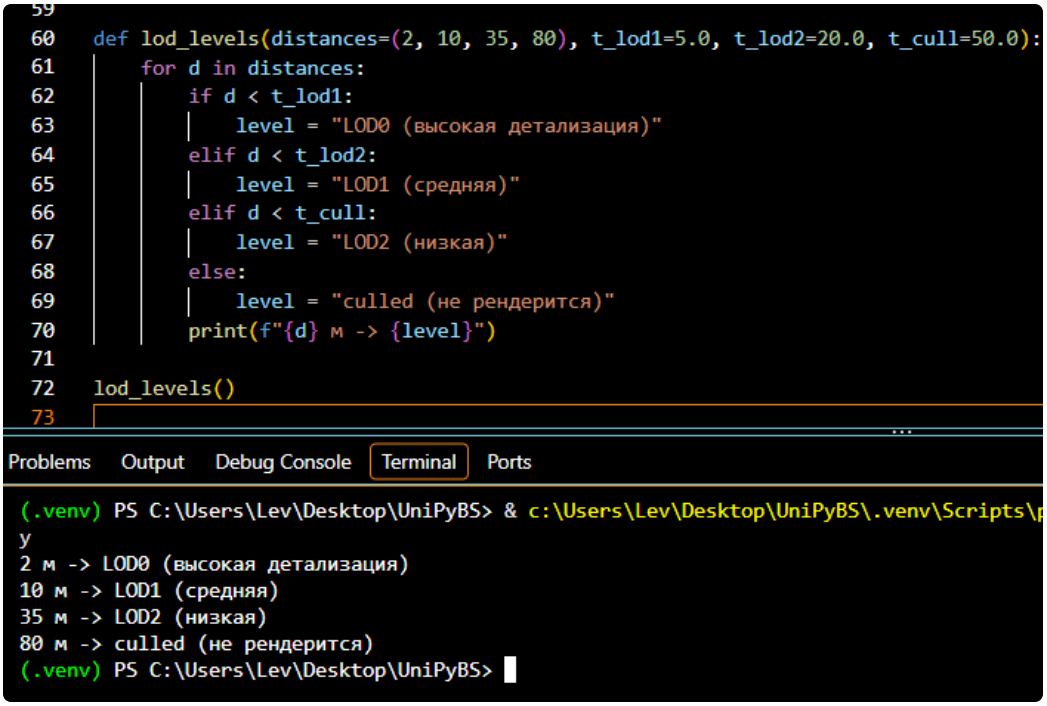
Task 12



Code:

```
def frame_budget(frequencies=(72, 90, 120), render_time_ms=9.5):
    for freq in frequencies:
        budget_ms = 1000 / freq
        print(f"{freq} Гц -> бюджет кадра {budget_ms:.2f} мс")
    budget_90 = 1000 / 90
    fits = render_time_ms <= budget_90
    status = "успеваем" if fits else "НЕ успеваем"
    print(f"Рендер {render_time_ms} мс при 90 Гц ({budget_90:.2f} мс):
{status}")
```

Task 13



Code:

```
def lod_levels(distances=(2, 10, 35, 80), t_lod1=5.0, t_lod2=20.0,
t_cull=50.0):
    for d in distances:
        if d < t_lod1:
            level = "LOD0 (высокая детализация)"
        elif d < t_lod2:
            level = "LOD1 (средняя)"
        elif d < t_cull:
            level = "LOD2 (низкая)"
        else:
            level = "culled (не рендерится)"
        print(f"{d} м -> {level}")
```

Task 14.

```
73  def check_polygon_budget(scene_budget=1_000_000, tris_per_object=8_000, object_count=150):
74      max_objects = scene_budget // tris_per_object
75      total_load = object_count * tris_per_object
76      fits = total_load <= scene_budget
77      print(f"Бюджет сцены: {scene_budget:,} тр., на объект: {tris_per_object:,} тр.")
78      print(f"Максимум объектов в бюджете: {max_objects}")
79      status = "в бюджете" if fits else "ПРЕВЫШЕН бюджет"
80      print(f"Сцена из {object_count} объектов = {total_load:,} тр. -> {status}")
81
```

Problems Output Debug Console Terminal Ports

```
(.venv) PS C:\Users\Lev\Desktop\UniPyBS> & c:\Users\Lev\Desktop\UniPyBS\.venv\Scripts\python.exe
y
Бюджет сцены: 1,000,000 тр., на объект: 8,000 тр.
Максимум объектов в бюджете: 125
Сцена из 150 объектов = 1,200,000 тр. -> ПРЕВЫШЕН бюджет
(.venv) PS C:\Users\Lev\Desktop\UniPyBS>
```

Code:

```python
def check_polygon_budget(scene_budget=1_000_000, tris_per_object=8_000,
object_count=150):
    max_objects = scene_budget // tris_per_object
    total_load = object_count * tris_per_object
    fits = total_load <= scene_budget
    print(f"Бюджет сцены: {scene_budget:,} тр., на объект: {tris_per_object:,}
тр.")
    print(f"Максимум объектов в бюджете: {max_objects}")
    status = "в бюджете" if fits else "ПРЕВЫШЕН бюджет"
    print(f"Сцена из {object_count} объектов = {total_load:,} тр. ->
{status}")
```

Task 15.

```
86
87  def check_motion_to_photon_latency(tracking_ms=2.0, processing_ms=3.0, render_ms=11.0, display_ms=4.0, threshold_ms=20.0):
88      total_latency = tracking_ms + processing_ms + render_ms + display_ms
89      is_comfortable = total_latency < threshold_ms
90      print(f"Датчик (трекинг): {tracking_ms} мс")
91      print(f"Обработка: {processing_ms} мс")
92      print(f"Рендер: {render_ms} мс")
93      print(f"Вывод на дисплей: {display_ms} мс")
94      print(f"Суммарная задержка motion-to-photon: {total_latency} мс")
95      print(f"Комфортно (< {threshold_ms} мс)? {is_comfortable}")
96
```

roblems Output Debug Console Terminal Ports

```
(.venv) PS C:\Users\Lev\Desktop\UniPyBS> & c:\Users\Lev\Desktop\UniPyBS\.venv\Scripts\python.exe c:/Users/Lev/Desktop/UniPyBS/vs_
y
Датчик (трекинг): 2.0 мс
Обработка: 3.0 мс
Рендер: 11.0 мс
Вывод на дисплей: 4.0 мс
Суммарная задержка motion-to-photon: 20.0 мс
Комфортно (< 20.0 мс)? False
(.venv) PS C:\Users\Lev\Desktop\UniPyBS>
```

Code:

```
def check_motion_to_photon_latency(tracking_ms=2.0, processing_ms=3.0,
render_ms=11.0, display_ms=4.0, threshold_ms=20.0):
    total_latency = tracking_ms + processing_ms + render_ms + display_ms
    is_comfortable = total_latency < threshold_ms
    print(f"Датчик (трекинг): {tracking_ms} мс")
    print(f"Обработка: {processing_ms} мс")
    print(f"Рендер: {render_ms} мс")
    print(f"Вывод на дисплей: {display_ms} мс")
    print(f"Суммарная задержка motion-to-photon: {total_latency} мс")
    print(f"Комфортно (< {threshold_ms} мс)? {is_comfortable}")
```